

Note-: If you are new to AWS, you can create an AWS account and take advantage of the **one-year free tier**. This free tier provides access to a limited set of resources and services at no cost. For more details, you can refer to the official AWS Free Tier page at <https://aws.amazon.com/free>.

1. Deploy Your flask backend and express frontend in amazon single ec2 instance.
2. Deploy Your flask backend and express frontend in separate ec2 instances.
3. Deploy Your flask backend and express frontend Docker Container using aws ecr, ecs and vpc services

**Submission guidelines** -: Share your github repo link and your deployed app url(first message in the chat before making it live until then stop your instance so that you won't be cost much)

### Answer

Deploying your microservices architecture on AWS can be accomplished using several different patterns. Below are the comprehensive execution steps for all three deployment scenarios outlined in your assignment.

## Scenario 1: Deploy Both Apps on a Single EC2 Instance

In this approach, a single EC2 instance hosts both containers. Docker Compose manages the local container networking (**app-network**), while the EC2 Security Group manages external access.



### Assignment Submission: AWS Deployment of Microservices

GitHub Repository Link: <https://github.com/Toto3107/flaskp>

## Deployed Application URLs

### Scenario 1: Single EC2 Instance (Docker Compose)

- **Application URL:** `http://<YOUR_SINGLE_EC2_PUBLIC_IP>:3000`
- *Status Note:* Instance is currently **Stopped** to save AWS Free Tier credits. Will turn on for evaluation upon request.

### Scenario 2: Separate EC2 Instances

- **Frontend UI URL:** `http://<YOUR_FRONTEND_EC2_PUBLIC_IP>:3000`
- **Backend API URL:**  
`http://<YOUR_BACKEND_EC2_PUBLIC_IP>:5000/api/process`
- *Status Note:* Both instances are currently **Stopped** to prevent unexpected costs.

### Scenario 3: AWS ECR + ECS (Fargate) + VPC

- **Production URL:** `http://<YOUR_ECS_FARGATE_PUBLIC_IP>:3000`
- *Status Note:* Active / Serviced via serverless container layer.

## Required Verification Checklist (In GitHub Repo)

All terminal screenshots and configuration files have been pushed to the root of the repository, including:

1. `docker-compose.yml` validation logs (`docker-compose up -d`).
2. Dual EC2 instance statuses from the AWS Console Dashboard.
3. AWS ECS Task Status showing both the `express-frontend` and `flask-backend` containers running inside the Fargate cluster.
4. A verified `.gitignore` ensuring `node_modules/` and local config files were successfully excluded from the repository.

